

Binary Tree - 6 Important Questions

class Node:

def __init__(self, data):

self.data = data

self.left = None

self.right = None

1. Print Boundary Nodes

def boundary(root):

if root is None:

return

print(root.data, end=" ")

def left_boundary(node):

while node:

if node.left or node.right:

print(node.data, end=" ")

if node.left:

node = node.left

else:

node = node.right

def leaves(node):

if node is None:

return

if node.left is None and node.right is None:

print(node.data, end=" ")

return

leaves(node.left)

leaves(node.right)

def right_boundary(node):

arr = []

while node:

if node.left or node.right:

```

        arr.append(node.data)
    if node.right:
        node = node.right
    else:
        node = node.left
    for i in range(len(arr) - 1, -1, -1):
        print(arr[i], end=" ")
    left_boundary(root.left)
    leaves(root.left)
    leaves(root.right)
    right_boundary(root.right)
def leftView(root):
    if root is None:
        return
    queue = [root]
    while queue:
        n = len(queue)
        for i in range(n):
            node = queue.pop(0)
            # First node of every level
            if i == 0:
                print(node.data, end=" ")
            if node.left:
                queue.append(node.left)

            if node.right:
                queue.append(node.right)
def rightView(root):
    if root is None:
        return

```

```

queue = [root]
while queue:
    n = len(queue)
    for i in range(n):
        node = queue.pop(0)
        if i == n - 1:
            print(node.data, end=" ")
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

```

4. Lowest Common Ancestor (LCA)

```

def LCA(root, n1, n2):
    if root is None:
        return None
    # If current node is one of the values
    if root.data == n1 or root.data == n2:
        return root
    left = LCA(root.left, n1, n2)
    right = LCA(root.right, n1, n2)
    if left and right:
        return root
    if left:
        return left
    return right

```

5. Path from Root to Given Node

```

def findPath(root, target, path):
    if root is None:
        return False
    path.append(root.data)

```

```

if root.data == target:
    return True
if findPath(root.left, target, path):
    return True
if findPath(root.right, target, path):
    return True
path.pop()

return False

```

6. Construct Tree from Preorder and Inorder

```

def buildTree(preorder, inorder):
    if not preorder or not inorder:
        return None
    root_value = preorder[0]
    root = Node(root_value)
    index = inorder.index(root_value)
    root.left = buildTree(
        preorder[1:index + 1],
        inorder[:index]
    )
    root.right = buildTree(
        preorder [index + 1:],
        inorder[index + 1:]
    )
    return root

root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)

```

```
root.right.left = Node(6)
root.right.right = Node(7)
print("1. Boundary Nodes:")
boundary(root)
print()
print("2. Left View:")
leftView(root)
print()
print("3. Right View:")
rightView(root)
print()
print("4. Lowest Common Ancestor:")
ans = LCA(root, 4, 5)
if ans:
    print(ans.data)
else:
    print("LCA not found")
print("5. Path from Root to Node:")
path = []
if findPath(root, 5, path):
    print(path)
else:
    print("Node not found")
print("6. Construct Tree:")
preorder = [1, 2, 4, 5, 3, 6, 7]
inorder = [4, 2, 5, 1, 6, 3, 7]
new_root = buildTree(preorder, inorder)
print("Tree constructed successfully")
```